

M04.1.1.1 - Tokens and Tokenization

LLM fundamentals: how text becomes the small pieces a model can process.

1. The Big Idea

Human view

We see a sentence as words and meaning.

```
TEXT
-> TOKENIZER
-> TOKENS
-> TOKEN IDS
-> MODEL
```

Core idea: an LLM does not directly process raw text as humans do. A tokenizer first breaks text into manageable pieces called tokens, then maps those pieces to numeric token IDs.

2. The Five Core Terms

Term	Simple meaning
Token	One small piece of text.
Vocabulary	The collection of token pieces the tokenizer can use.
Tokenization	The process of splitting text into tokens.
Token ID	The number used to represent a token.
Token count	How many tokens a piece of text contains.

3. Tokens

A token is not always a whole word. It can be a whole word, part of a word, punctuation, or another text fragment. Exact token boundaries depend on the tokenizer.

```
"I love coding!"

Possible token view:
I | love | coding | !
```

Remember: 1 word does NOT necessarily equal 1 token.

M04.1.1.1 - Subword Tokenization & BPE

Why models split words into reusable pieces instead of memorizing every possible word.

4. Vocabulary

The vocabulary is the tokenizer's inventory of available token pieces. A vocabulary can contain complete words as well as smaller fragments.

Example vocabulary pieces:
play | er | s | ed | ing | fully | !

Why vocabulary matters

A finite set of reusable pieces lets the tokenizer represent a huge number of words, including words it has not stored as single tokens.

5. Subword Tokenization

Subword tokenization sits between character-level and word-level tokenization.

Approach	Example	Idea
Character	c a t	Very small pieces; long sequences.
Word	cat	Simple for known words; weak for rare/unseen words.
Subword	play ing	Reusable pieces; good balance.

6. BPE - Byte Pair Encoding Intuition

BPE is a common idea for learning useful, reusable pieces from text. In simplified intuition, frequent neighboring pieces can be merged into a larger token.

Start with small pieces
p | l | a | y | i | n | g

Frequent combinations can become useful pieces
play + ing

Result:
play | ing

Why this works

Instead of storing every possible word as a unique item, the tokenizer can reuse pieces such as "play" and "ing" across many words: playing, played, player, etc.

Interview intuition: subword tokenization gives a finite reusable vocabulary while still allowing rare or unseen words to be represented by combinations of smaller pieces.

M04.1.1.1 - Token Counts, Cost & Revision

The practical side: context windows, API billing, computation, and interview-ready recall.

7. Token Count

The token count is the number of tokens produced from text. It varies by tokenizer and can change substantially for code, numbers, punctuation, unusual words, or different languages.

"Artificial intelligence is changing software."

Conceptual token view:

Artificial | intelligence | is | changing | software | .

Rule of thumb for English: a rough mental estimate is around 1 token per 0.75 words (or about 130 tokens per 100 words). This is only an estimate.

8. Why Token Count Matters

Factor	Effect of more tokens
API cost	Many LLM APIs bill based on input and/or output tokens, so more tokens can mean higher cost.
Context window	More tokens consume available context, leaving less room for other content and output.
Computation	More tokens generally require more processing work.
Latency	Larger inputs/outputs can increase the amount of work and therefore affect response time.

9. The Full LLM Text Pipeline

Raw text -> Tokenizer -> Tokens -> Token IDs -> Embeddings
-> Transformer / Model -> Next-token prediction
-> Generated tokens -> Decoded text

10. Quick Revision Table

Question	Answer
What is a token?	A small piece of text processed by the model.
What is vocabulary?	The collection of token pieces available to the tokenizer.
What is tokenization?	Breaking text into tokens and mapping them to token IDs.
Why subwords?	They reuse pieces and handle rare/unseen words efficiently.
Why count tokens?	Token count affects context usage, computation, latency, and often API cost.

11. Memory Trick

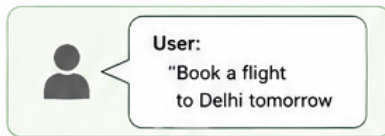
TEXT -> TOKENS -> IDs -> MODEL

Token = piece | Vocabulary = collection | Tokenization = splitting | BPE = reusable pieces | Token count = cost + context

HOW AN LLM PROCESSES YOUR INPUT AND GENERATES OUTPUT

1 USER INPUT

You type or speak something.

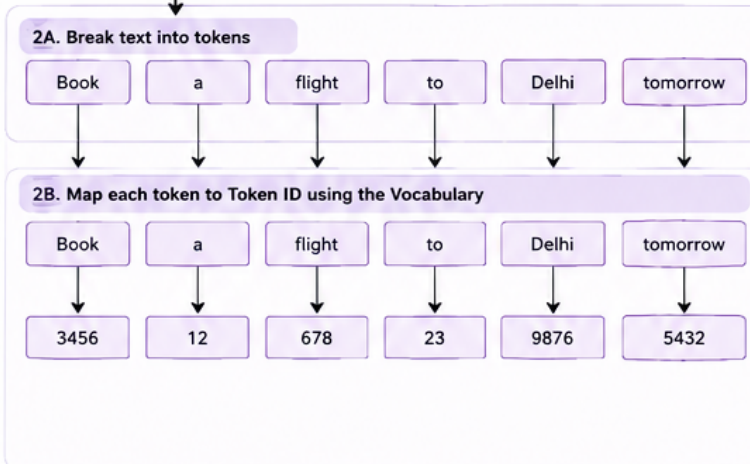


What is happening?

You give a prompt (text) to the LLM.

2 TOKENIZATION (Using Vocabulary)

The text is split into tokens (small pieces) using the model's vocabulary.



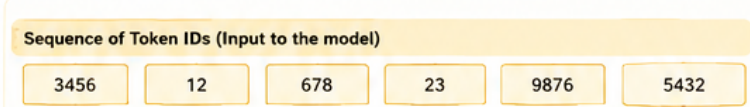
What is happening?

- The text is split into tokens. (Tokens can be words, parts of words, or even characters.)
- Each token is looked up in the vocabulary (a big list of all possible tokens the model knows).
- Each token gets a unique number called **Token ID**.

Vocabulary (Example)	
Token	Token ID
Book	3456
a	12
flight	678
to	23
Delhi	9876
tomorrow	5432
...	...

3 TOKEN IDS

Now we have a sequence of numbers (Token IDs).

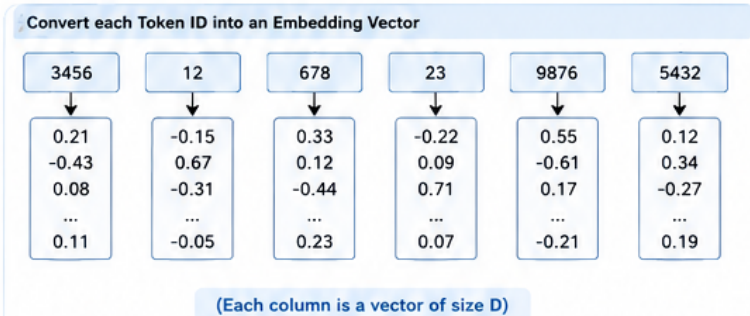


What is happening?

These numbers represent your input for the model.

4 EMBEDDING LOOKUP

Each Token ID is converted into a vector (embedding) using an Embedding Matrix.



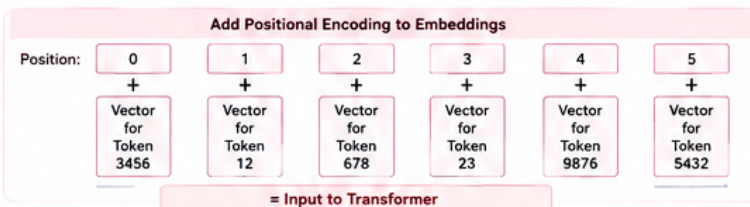
What is happening?

- The Embedding Matrix is a big table learned during training.
- It converts each Token ID into a dense vector (list of decimal numbers).
- These vectors capture the meaning of the tokens.

Example
If embedding size (D) = 4
Then each token becomes a 4-dimensional vector.
(Real models use D = 768, 1024, 4096 ... etc.)

5 ADD POSITION INFORMATION

Since transformers don't know order by default, we add positional encoding to embeddings.

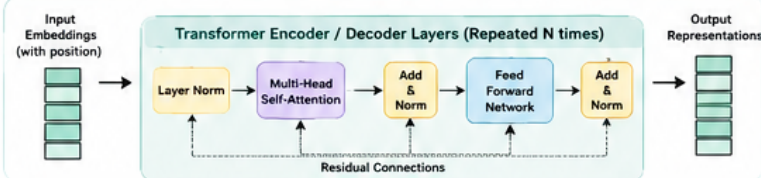


What is happening?

We add position info to each vector so the model knows the order of tokens.

6 TRANSFORMER (SELF-ATTENTION PROCESSING)

The transformer processes all vectors together using self-attention across many layers.

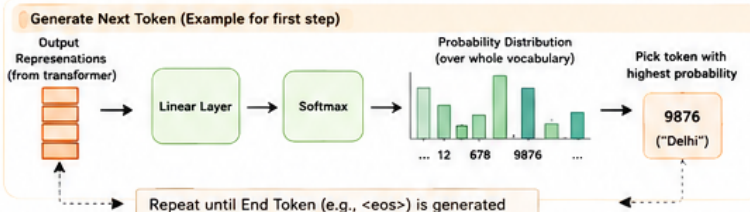


What is happening?

- Self-attention lets the model look at all words and understand relationships.
- This happens in multiple layers.
- The output is a new set of vectors that understand the context.

7 PREDICT NEXT TOKEN (AUTO-REGRESSIVE DECODING)

The model predicts the next token one at a time.

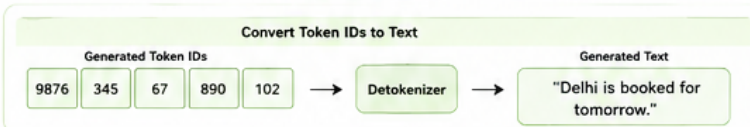


What is happening?

- The model converts the output to probabilities over the entire vocabulary.
- It picks the most likely next token (or uses sampling strategies).
- That token is appended and the process repeats.

8 DETOKENIZATION (OUTPUT TO TEXT)

The generated token IDs are converted back to text.



What is happening?

The sequence of generated Token IDs is converted back into human-readable text and shown to you.

QUICK RECAP



Notes:

- Vocabulary size: typically 30k - 200k tokens
- Embedding size (D): 512, 768, 1024, 4096...
- Number of layers: 6, 12, 24, 32, 70... etc.